

Lecture 10: Reinforcement Learning
Bellman Equation, Q-Learning,
Policy Gradient, Circuit Compilation

Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

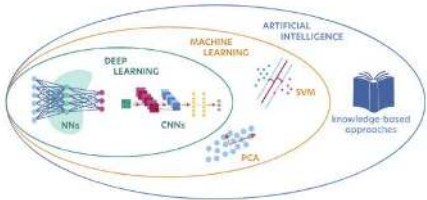
1

Learning from Rewards



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Three Paradigms of Machine Learning



Main ML paradigms:

1. Unsupervised Learning
2. Supervised Learning

Teacher-Student scheme

3. Reinforcement Learning

Finding best strategy

- supervised learning imitates a teacher, the labelled data
- reinforcement learning has no teacher, only a scalar reward for what the agent does

Go Is Judged Only at the End of the Game

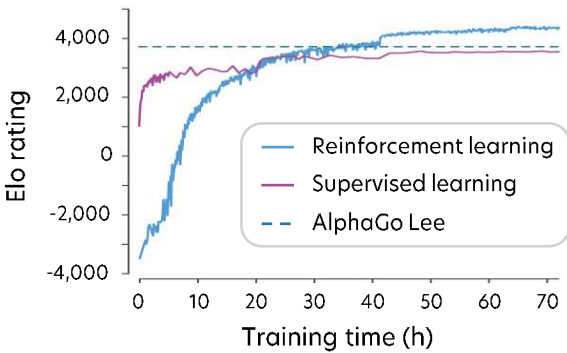


AlphaGo against Lee Sedol, Seoul, March 2016

- 19×19 board, $3^{361} \approx 10^{172}$ configurations, about 2×10^{170} of them legal
- a move is judged only by the result of the game
- 2015–2016: AlphaGo beats Fan Hui 5:0 and Lee Sedol 4:1, trained first on expert games, then by self-play

D. Silver *et al.*, Nature **529**, 484 (2016); legal positions: J. Tromp, LNCS **10068**, 183 (2016)

AlphaGo Zero Learns Without Human Games



Elo rating: playing strength from game results; 400 points more wins about 91% of games

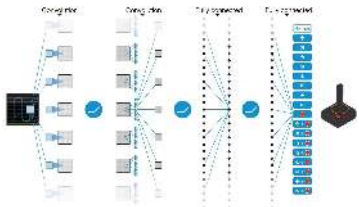
blue: AlphaGo Zero, self-play, given only the rules
purple: the same network trained on expert games
dashed: the version that beat Lee Sedol

The supervised start leads early and then limits the player. Self-play passes the Lee Sedol version after about 36 hours.

AlphaZero (2018) uses one algorithm for Go, chess and shogi. MuZero (2020) is not even given the rules, and learns a model of the environment to plan with.
D. Silver *et al.*, Nature **550**, 354 (2017); Science **362**, 1140 (2018); J. Schrittwieser *et al.*, Nature **588**, 604 (2020)

Video Games Learned from Play





- Atari (2015): one network for 49 games, input = screen frames, reward = change of the score
- StarCraft II (2019): AlphaStar at Grandmaster level, above 99.8% of ranked human players
- Dota 2 (2019): OpenAI Five defeats the world champion team

V. Mnih *et al.*, Nature **518**, 529 (2015); O. Vinyals *et al.*, Nature **575**, 350 (2019); C. Berner *et al.*, arXiv:1912.06680 (2019)

Hide and Seek Produces Strategies Nobody Programmed



click to open the video

- reward only for the game: hiders $+1$ if all are hidden, -1 if any is seen
- nothing in the reward mentions boxes or ramps
- shelters, ramps over walls and locked ramps appear one after another

B. Baker *et al.*, arXiv:1909.07528, ICLR (2020);
OpenAI blog post with videos

Robots Trained with Reinforcement Learning

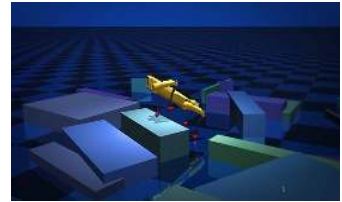
Recent robots of Boston Dynamics use locomotion policies learned with reinforcement learning, trained in simulation. Click a picture to open the video.



Walk, Run, Crawl, RL Fun (2025), Atlas, policies learned with RL from motion capture



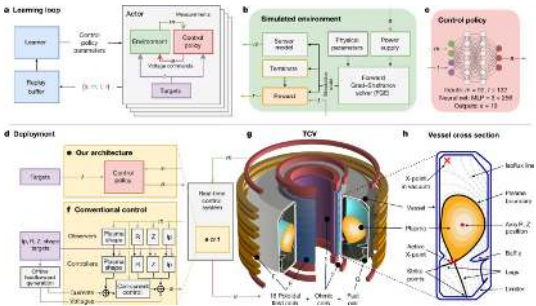
Atlas Airborne (2026), full-body control, acrobatics, failed attempts



Reinforcement learning with Spot, policy trained in a million simulations

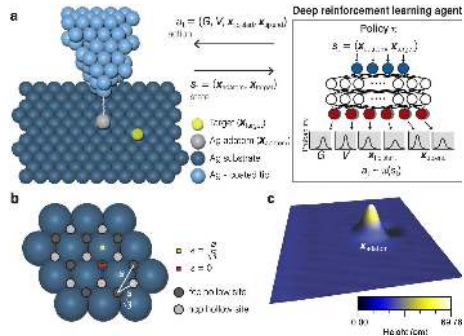
Boston Dynamics, with the RAI Institute; Boston Dynamics blog on reinforcement learning with Spot

Reinforcement Learning in the Laboratory



Fusion plasma: a policy network trained only in a simulator of the TCV tokamak (top: learning loop) sets the voltages of the control coils in real time and shapes the plasma (bottom)

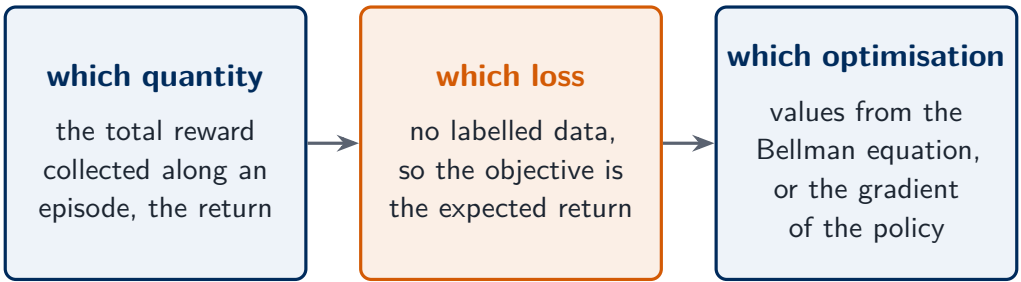
J. Degrave *et al.*, Nature **602**, 414 (2022), Fig. 1



Single atoms: state = positions of the silver adatom and of its target; action = tip bias, current and start and end positions of the tip; the agent moves atoms into designed structures with a scanning tunnelling microscope

I.-J. Chen *et al.*, Nat. Commun. **13**, 7499 (2022), Fig. 1

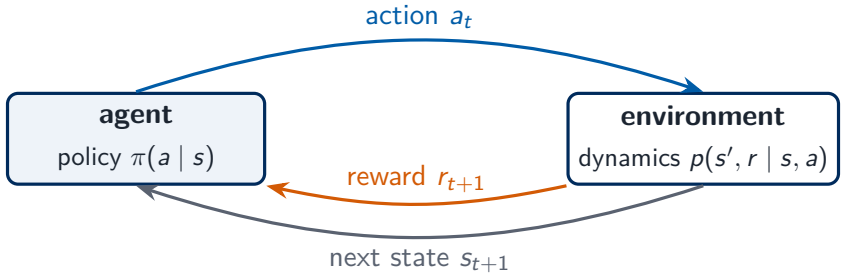
Three Questions Shape the Formalism



2
Building Blocks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

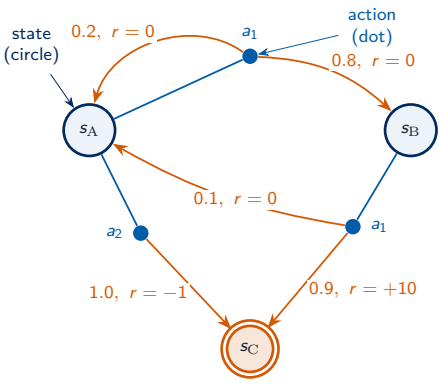
Agent and Environment



- the agent observes the state s_t and chooses an action a_t
- the action changes the state of the environment, $s_t \to s_{t+1}$
- the environment returns a reward r_{t+1}, a single number

time runs in discrete steps $t = 0, 1, 2, \ldots$

States and Actions



illustration; double circle: terminal state

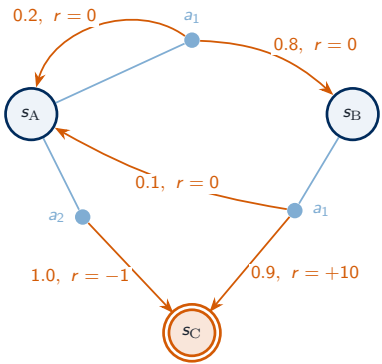
State space and action space

$\mathcal{S} :=$ the set of all states of the environment
$\mathcal{A}(s) :=$ the set of actions available in the state s

- Go: a position of the stones; a legal move
- robot arm: joint angles and velocities; joint torques
- a controller: the observed system state; an available control

finite, as in Go, or continuous, as for the robot

The Reward



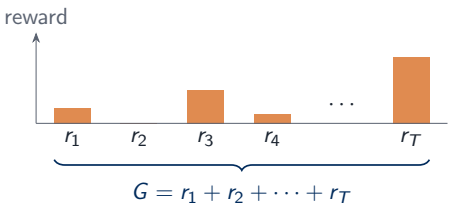
illustration; every outcome carries a reward r

Reward

$r_{t+1} \in \mathbb{R}$, one number after every action, given by the environment

- Go: $+1$ for a win, -1 for a loss, 0 for every move before the end
- the reward rates what happened, never which action was right

The Goal Is the Largest Total Reward



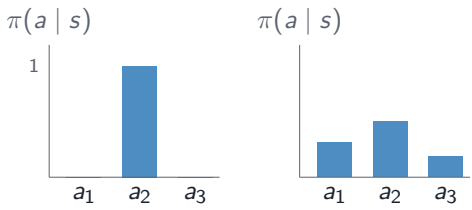
the rewards of one episode, illustration

Return of an episode

$$G \coloneqq r_1 + r_2 + \cdots + r_T$$

- T: the last step of the episode
- goal: actions that make G as large as possible on average
- a step without reward can prepare a large reward later

The Policy Is the Strategy of the Agent



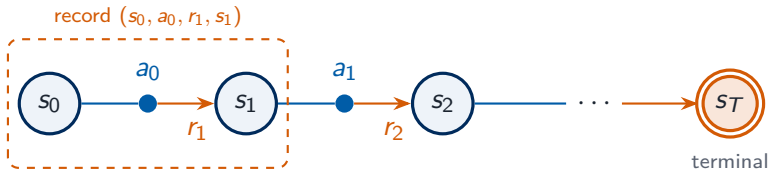
one state s, three actions, illustration

Policy

$\pi(a \mid s) :=$ the probability of choosing action a in state s

- deterministic policy: one action per state
- random policy: $\pi(a \mid s) = 1/|\mathcal{A}|$
- learning = finding a policy with a large return

One Episode Is a Trajectory with a Return



- trajectory $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$, an episode ends in a terminal state
- the agent records transitions $(s_t, a_t, r_{t+1}, s_{t+1})$, nothing else
- which earlier action earned a late reward: the credit assignment problem

the length T is random; the plain sum G does not prefer a faster agent

3

Markov Decision Processes

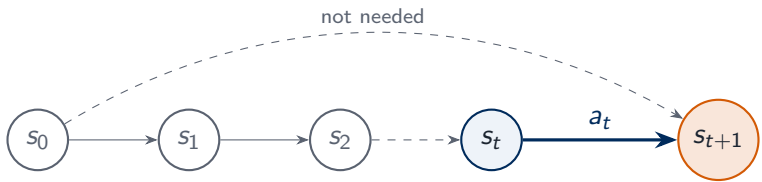
JAGIELLONIAN UNIVERSITY
IN KRAKOW

The Present State Is Enough

the future depends on the past only through the present state

Markov property

$$p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1} \mid s_t, a_t)$$



- the state is a sufficient statistic for the future
- an environment with memory: put the remembered information into the state

The Dynamics Is the Model of the Environment

where the agent lands and which reward it receives

Dynamics

$$p(s', r \mid s, a) \coloneqq \Pr\left(s_{t+1} = s',\ r_{t+1} = r \mid s_t = s,\ a_t = a\right)$$

- $\sum_{s',r} p(s', r \mid s, a) = 1$ for every s and a
- deterministic environment: p is 0 or 1
- next-state probability $p(s' \mid s, a) = \sum_r p(s', r \mid s, a)$
- the agent usually does not know p, it sees samples (s, a, r, s')

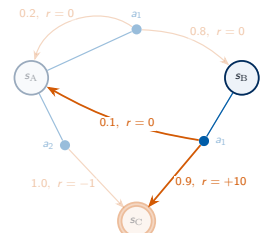


illustration: in s_B the action a_1 gives $(s_\mathrm{C}, +10)$ with 0.9 and $(s_\mathrm{A}, 0)$ with 0.1

A Markov Decision Process Collects the Ingredients

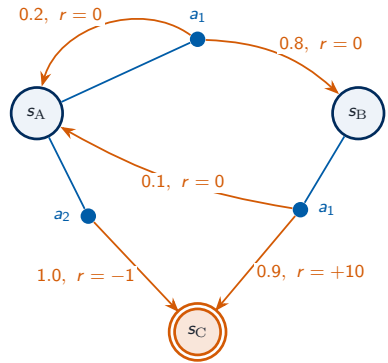
Markov decision process (MDP)

$$(\mathcal{S},\ \mathcal{A},\ p,\ r,\ \gamma)$$

- $\mathcal{S}$, $\mathcal{A}$: states and actions; p: dynamics
- r: reward, the definition of the task
- $\gamma \in [0, 1]$: discount factor, how much the future counts

decision: the action of the agent changes the probabilities of what comes next

A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)



illustration; circles: states, dots: actions, arrows: $p(s', r \mid s, a)$, probability and reward

The Discounted Return Prefers Early Rewards

Discounted return

$$G_t := \sum_{k=0}^{T-t-1} \gamma^k \, r_{t+k+1} = r_{t+1} + \gamma \, G_{t+1}$$

- a reward k steps ahead counts with γ^k
- $\gamma < 1$: finite G_t even for $T = \infty$, horizon about $1/(1-\gamma)$
- $\gamma \to 0$ myopic, $\gamma \to 1$ long-term

rewards $0, 0, 0, 10$, $\gamma = 0.9$: $G_3 = 10$, $G_2 = 9$, $G_1 = 8.1$, $G_0 = 7.29$



(a) a myopic agent settles for early small rewards; (b) a long-term agent accepts poor early steps for a larger later reward

A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)

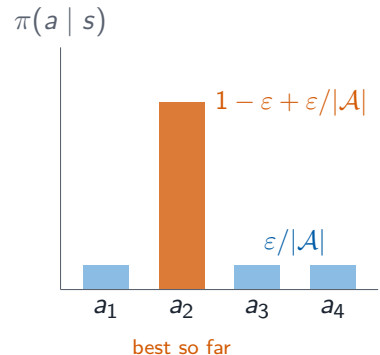
Exploration and Exploitation

ε-greedy policy

a random action with probability ε, otherwise the best action known so far

- exploit: use what worked
- explore: try actions that may be better
- $\varepsilon = 0$ commits to the first strategy that works

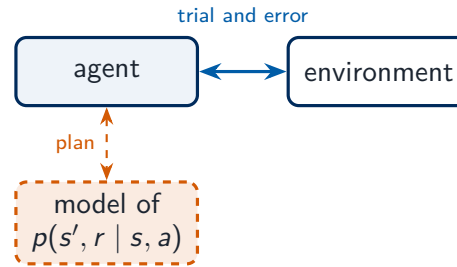
common practice: large ε at the start, lowered during training



ε-greedy policy in one state with four actions; a_2 has the largest estimated value; every action keeps a probability of at least $\varepsilon/|\mathcal{A}|$

Model-Free and Model-Based Reinforcement Learning

- **model-based**: knows $p(s', r \mid s, a)$ or learns it, and plans ahead
- **model-free**: learns from sampled transitions (s, a, r, s') by trial and error
- a model helps when interaction is costly, as in an experiment



MuZero learns a model; a laboratory controller usually sees outcomes only

A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)

4

Values and the Bellman Equation

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Value Functions Let Us Compare Policies

A policy $\pi(a \mid s)$ specifies the action probabilities. Its value functions give the expected discounted return G_t when we follow that policy.

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] \qquad \text{start in } s \text{ and follow } \pi,$$
$$Q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] \qquad \text{take } a, \text{ then follow } \pi,$$
$$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s) \qquad \text{gain over the policy's average at } s.$$

π_1 is at least as good as π_2 if $V_{\pi_1}(s) \geq V_{\pi_2}(s)$ in every state s. If each policy is better in different states, neither dominates.

Running example, $\gamma = 0.9$, policy taking a_1 in s_A

$V_\pi(s_\mathrm{A}) = 8.58$, $V_\pi(s_\mathrm{B}) = 9.77$, and $Q_\pi(s_\mathrm{A}, a_2) = -1$, so $A_\pi(s_\mathrm{A}, a_1) = 0$ and $A_\pi(s_\mathrm{A}, a_2) = -9.58$.

The Bellman Equation Follows from the Return

The discounted return separates into the reward now and the return afterwards,

$$G_t = r_{t+1} + \gamma G_{t+1}.$$

Insert this into the definition of state value,

$$V_\pi(s) = \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s].$$

The policy chooses a, then the environment produces (s', r),

$$V_\pi(s) = \sum_a \underbrace{\pi(a \mid s)}_{\text{action probability}} \sum_{s',r} \underbrace{p(s', r \mid s, a)}_{\text{outcome probability}} \left[r + \gamma \underbrace{\mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s']}_{\text{expected return after reaching } s'} \right].$$

The Markov property makes s' sufficient for the future. Following the same policy from s', the underbraced expectation is $V_\pi(s')$. Therefore

$$\boxed{V_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[\underbrace{r}_{\text{reward now}} + \underbrace{\gamma V_\pi(s')}_{\text{discounted future return}} \right].}$$

The Model Lets Us Compute an Optimal Policy

An optimal policy π^* has the largest expected return from every state. Its value is $V_*(s) = \max_\pi V_\pi(s)$. Choosing the best action gives

$$V_*(s) = \max_a \underbrace{\sum_{s',r} p(s', r \mid s, a)[r + \gamma V_*(s')]}_{Q_*(s,a),\ \text{expected return for action } a}.$$

Once these values are known, we choose

$$\pi^*(s) \in \arg\max_a Q_*(s, a),$$

so an optimal policy can always be chosen greedy with respect to Q_*.

Known environment model

We compute the average for each action and solve the coupled equations by value iteration.

Unknown environment model

We update estimates from observed transitions (s, a, r, s'), as in Q-learning.

Value Iteration Needs the Model

Value iteration

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_k(s')\big]$$

$$\|V_k - V_*\|_\infty \le \gamma^k \|V_0 - V_*\|_\infty$$

- one nonlinear equation per state, needs p
- for $\gamma < 1$ the error shrinks at least by γ per sweep

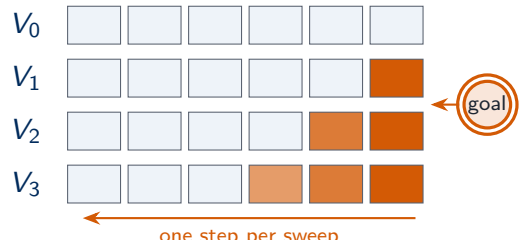


illustration: a chain of six states with the reward at its right end, $V_0 = 0$; coloured: states whose value V_k already contains the reward; the k-th sweep reaches the state k steps from the goal

5

Learning Without a Model

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Learning a Value from Experience

For a fixed policy π we want $V_\pi(s_t) = \mathbb{E}_\pi[G_t \mid s_t]$, but without a model we only see sampled episodes. We keep a learned estimate V, distinct from the unknown V_π. An *update target* y_t is a number computed from experience that stands in for $V_\pi(s_t)$.

Monte Carlo target

Complete the episode and use its observed return,

$$y_t = G_t.$$

An unbiased sample of $V_\pi(s_t)$, available only at the end of the episode and noisy through every later step.

Temporal-difference (TD) target

Observe one transition and its reward,

$$y_t = \underbrace{r_{t+1}}_{\text{observed}} + \gamma \underbrace{V(s_{t+1})}_{\text{estimated}}.$$

Available after one step, but it inherits the error of the current estimate.

Both rules move the estimate a fraction α towards the target,

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[y_t - V(s_t)]}_{\text{correction to the estimate}}.$$

$0 < \alpha \leq 1$ is the learning rate. TD uses the next-state estimate, called bootstrapping. With $\alpha = 1/n$ at the n-th visit Monte Carlo is a sample average and converges to V_π; a constant α leaves fluctuations.

SARSA and Q-Learning

the TD update for action values

SARSA, on-policy

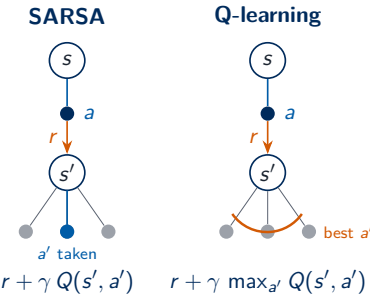
$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]$$

Q-learning, off-policy

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

- SARSA: a' from the acting policy, learns its Q_π
- Q-learning: learns Q_* while acting ε-greedily, if every pair (s,a) keeps being tried

C. J. C. H. Watkins, P. Dayan, Mach. Learn. **8**, 279 (1992)



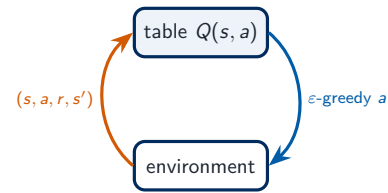
after the transition (s,a,r,s'): SARSA uses the next action a' that the agent takes, Q-learning the best next action

The Q-Learning Algorithm

1. initialise $Q(s, a)$ for all s, a
2. start an episode in s_0
3. ε-greedy: random a with probability ε, else $\arg\max_a Q(s, a)$
4. take the action, observe s' and reward r
5. $Q(s, a) \leftarrow Q(s, a)$
   $+\alpha\big(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big)$
6. $s \leftarrow s'$, repeat until the episode ends, then start another episode

At a terminal state the future value in the target is zero.

A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)



one observed transition, one updated table entry

A Q-Table Changes One Entry per Transition

running example: rows are states, columns are actions, $\gamma = 0.9$, $\alpha = 0.2$

before

	a_1	a_2
s_A	2	-1
s_B	3	–

entries learned so far (illustration); s_B has no a_2, terminal s_C has value 0

transition $(s_B, a_1, 10, s_C)$

	a_1	a_2
s_A	2	-1
s_B	**4.4**	–

$$y = 10 + 0$$

reward, no continuation at the terminal state

$$3 + 0.2\,(10 - 3) = 4.4$$

transition $(s_A, a_1, 0, s_B)$

	a_1	a_2
s_A	**2.392**	-1
s_B	4.4	–

$$y = 0 + 0.9 \max_{a'} Q(s_B, a') = 0.9 \cdot 4.4 = 3.96$$

uses the entry just updated

$$2 + 0.2\,(3.96 - 2) = 2.392$$

Each transition changes one entry. The reward reaches s_A through the bootstrapped target.

Q-Learning Follows the Optimality Residual

The sampled target uses the best estimated continuation,

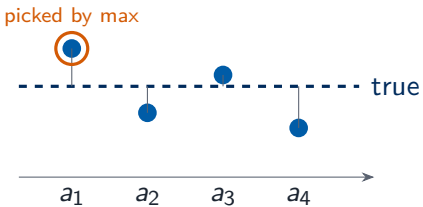
$$Y_t = r_{t+1} + \gamma \max_{a'} Q_t(s_{t+1}, a'), \qquad \delta_t^Q = Y_t - Q_t(s_t, a_t).$$

For a fixed visited pair and current table,

$$\mathbb{E}[\delta_t^Q \mid s_t = s, a_t = a, Q_t] = \underbrace{\sum_{s',r} p(s', r \mid s, a)[r + \gamma \max_{a'} Q_t(s', a')]}_{\text{Bellman optimality target}} - Q_t(s, a).$$

- Exploration chooses which pairs supply data. The target still uses a greedy maximum.
- The observed error supplies a correction without knowing $p(s', r \mid s, a)$.
- Tabular convergence needs every pair visited repeatedly and decreasing step sizes. A neural network requires separate evaluation.

Double Q-Learning Separates Choice and Evaluation



two equal values, Gaussian noise σ_e:

$$\mathbb{E}\max = \sigma_e/\sqrt{\pi} \approx 0.56\,\sigma_e$$

Double Q-learning update, two tables Q^A, Q^B

$$Q^A(s,a) \leftarrow Q^A(s,a) + \alpha\big[y - Q^A(s,a)\big]$$
$$y = r + \gamma\, Q^B\big(s', \arg\max_{a'} Q^A(s',a')\big)$$

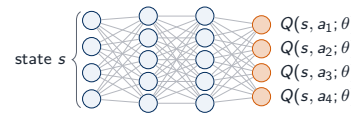
- the maximum of noisy estimates is biased upwards
- one table chooses the action, the other evaluates it
- roles of A and B exchanged at random

H. van Hasselt, Adv. Neural Inf. Process. Syst. **23** (2010)

A Network Replaces the Table

A deep Q-network (DQN) takes state features as input and outputs an estimated return for each allowed action.

We train its parameters θ by regression on sampled transitions.



$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^{B} [y_i - Q(s_i, a_i; \theta)]^2$$

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q(s_i', a'; \theta^-)$$

B is the batch size. The flag $d_i = 1$ marks a terminal next state. θ^- is a target-network copy, explained next.

V. Mnih et al., Nature 518, 529 (2015).

Experience Replay and Double DQN

Store transitions (s, a, r, s', d) in a replay buffer. Random batches reduce correlations between consecutive samples and reuse collected experience.

The online network $Q(s, a; \theta)$ is trained. A target network with parameters θ^- is copied from it periodically and held fixed between copies.

Double DQN separates selection and evaluation

$$a^* = \underbrace{\arg\max_{a'} Q(s', a'; \theta)}_{\text{online network selects an action}}, \qquad y = r + \gamma(1 - d)\ \underbrace{Q(s', a^*; \theta^-)}_{\text{target network evaluates it}}.$$

We fit $Q(s, a; \theta)$ to the update target y, keeping y fixed during differentiation.

V. Mnih et al., Nature 518, 529 (2015). H. van Hasselt, A. Guez, D. Silver, Proc. AAAI 30 (2016).

The Expected Return and Its Policy Gradient

We optimise a differentiable policy $\pi_\theta(a \mid s)$. A trajectory τ has discounted return $G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$.

$$\mathcal{J}(\theta) = \sum_\tau p_\theta(\tau) G(\tau), \qquad p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

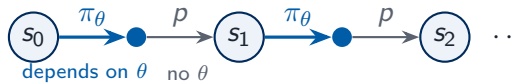
For a fixed trajectory, its return is fixed. The gradient changes its probability. Use $\nabla_\theta p_\theta = p_\theta \nabla_\theta \ln p_\theta$,

$$\begin{aligned} \nabla_\theta \mathcal{J} &= \sum_\tau G(\tau) \nabla_\theta p_\theta(\tau) \\ &= \sum_\tau p_\theta(\tau) G(\tau) \nabla_\theta \ln p_\theta(\tau) \\ &= \mathbb{E}_{\tau \sim p_\theta}[G(\tau) \nabla_\theta \ln p_\theta(\tau)]. \end{aligned}$$

We estimate this average using episodes generated by the current policy, then take a gradient-ascent step to increase the expected return.

The Dynamics Drops Out

$$\ln p_\theta(\tau) = \sum_t \ln \pi_\theta(a_t \mid s_t) + \sum_t \ln p(s_{t+1} \mid s_t, a_t)$$



Policy-gradient theorem

$$\nabla_\theta \mathcal{J} = \mathbb{E}\Big[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \ln \pi_\theta(a_t \mid s_t) \Big]$$

no model of the environment and no derivative of the reward needed; $\nabla_\theta \ln \pi_\theta(a \mid s)$ is the score function

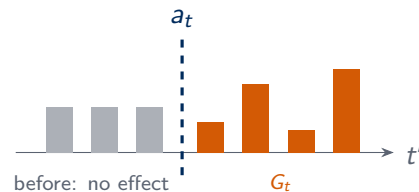
R. S. Sutton *et al.*, Adv. Neural Inf. Process. Syst. **12** (2000)

Only Rewards After an Action Reinforce It

write $G(\tau) = \sum_{t'} \gamma^{t'} r_{t'+1}$ and drop the terms with $t' < t$, whose average vanishes:

Reward-to-go form

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}\Big[\sum_{t=0}^{T-1} \gamma^t \, G_t \, \nabla_\theta \ln \pi_\theta(a_t \mid s_t) \Big]$$



- the past cannot be changed by a_t
- less noise, same mean

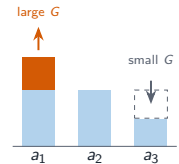
A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)

REINFORCE Reinforces What Worked

Sample B complete episodes with the current policy and use their returns,

$$\theta \leftarrow \theta + \frac{\alpha}{B} \sum_{i=1}^{B} G(\tau_i) \sum_t \nabla_\theta \ln \pi_\theta(a_t^{(i)} \mid s_t^{(i)})$$

- On average, actions associated with larger returns become more probable.
- A baseline $b(s_t)$ subtracted from the return leaves the mean unchanged. Whether it reduces the noise must be checked.



α is the learning rate and τ_i a sampled episode. In the reward-to-go form, without the factor γ^t the update is no longer the gradient of the discounted objective and weights late steps more strongly.

R. J. Williams, Mach. Learn. 8, 229 (1992).

A Policy Network Outputs Action Probabilities

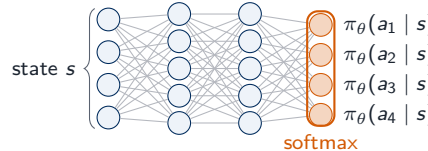
Softmax policy

$$\pi_\theta(a \mid s) = \frac{e^{z_\theta(s,a)}}{\sum_{a'} e^{z_\theta(s,a')}}$$

REINFORCE as a loss

$$\mathcal{L}(\theta) = -\frac{1}{B}\sum_{i=1}^{B} G(\tau_i) \sum_t \ln \pi_\theta\left(a_t^{(i)} \mid s_t^{(i)}\right)$$

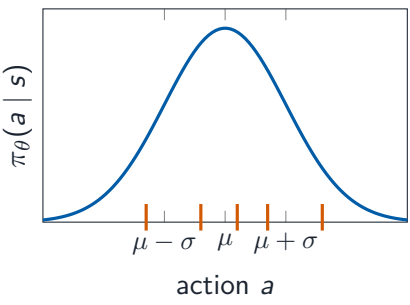
$z_\theta(s, a)$: action preferences (logits); the loss is a cross-entropy with the taken action as label and G as weight



sampled action $a \sim \pi_\theta(\cdot \mid s)$

A. Dawid *et al.*, arXiv:2204.04198; R. S. Sutton, A. G. Barto, *Reinforcement Learning* (MIT Press, 2018)

Continuous Actions Need a Gaussian Policy



sampled actions (orange) explore around the mean

Gaussian policy

$$\pi_\theta(a \mid s) = \frac{1}{\sqrt{2\pi}\,\sigma}\, e^{-(a-\mu)^2/2\sigma^2}$$

$$\partial_\mu \ln \pi = \frac{a-\mu}{\sigma^2}, \quad \partial_\nu \ln \pi = \frac{(a-\mu)^2}{\sigma^2} - 1$$

- a Q-network would need $\max_a$ over a continuum at every step
- mean $\mu_\theta(s)$, width $\sigma = e^\nu$ from the network
- a better-than-baseline action pulls μ towards it

The Actor Chooses and the Critic Estimates

Actor $\pi_\theta(a \mid s)$

Chooses an action from the current state. Its parameters θ define the policy.

Critic $V_\phi(s)$

Estimates the expected return of the actor's policy. It learns parameters ϕ.

$$\delta_t = \underbrace{r_{t+1} + \gamma V_\phi(s_{t+1})}_{\text{one-step target}} - \underbrace{V_\phi(s_t)}_{\text{current estimate}}$$

$$\theta \leftarrow \theta + \alpha_\theta \gamma^t \delta_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)$$

With an accurate critic, the conditional mean of δ_t is the advantage. The critic learns towards the same one-step target.

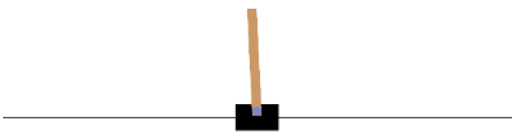
6

Classical Control



JAGIELLONIAN UNIVERSITY
IN KRAKOW

CartPole Turns Pushes Into Survival

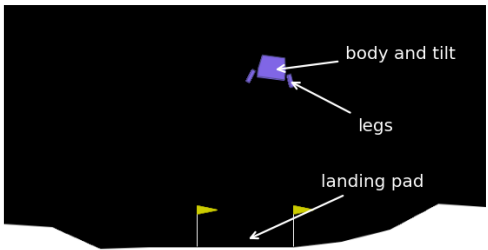


A pole hinged to a moving cart. The image illustrates the task; the actor reads numerical observations.

- Observation $s = (x, v, \vartheta, \omega)$ includes position, velocity, tilt and angular velocity.
- Action 0 pushes with 10 N to the left; action 1 to the right.
- Tilt grows unless the cart moves under the pole: $\ell\ddot{\vartheta} \approx g \sin\vartheta - \ddot{x}\cos\vartheta$.
- Reward $+1$ per step. Failure ends the physical task; the wrapper caps evaluation at 500 steps.

In a new evaluation of the saved actor all 20 reset seeds reach 500 steps.

LunarLander Chooses Engine Commands



The controller must reduce displacement, speed and tilt. The flags mark the nominal pad.

- Eight observations give position, velocity, orientation, rotation and two leg-contact flags.
- Four actions select no engine, the left orientation engine, the main engine or the right orientation engine.
- Reward: a shaping term for approaching the pad slowly and upright, minus 0.3 per main-engine step and 0.03 per side-engine step.
- Coming to rest ends the episode with $+100$, a crash with -100.

Action Values and Policy Probabilities Control the Same Lander

DQN

The network outputs four return estimates $Q(s, a)$. The controller selects $\arg\max_a Q(s, a)$.

PPO actor and critic

The actor outputs four action probabilities; we act with the most probable one. The critic estimated $V(s)$ during training.

saved controller	mean return	landings at rest
DQN	252.6	16 of 20
PPO actor	200.0	15 of 20
random actions	-166.2	0 of 20

Same 20 reset seeds; the training budgets differ, so this ranks the two controllers, not the algorithms.

7

Quantum State Preparation



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Constructing the Evolution to a Target State

We know the initial state $|\psi_0\rangle$ and a target $|\psi_{\text{target}}\rangle$. We seek an allowed unitary evolution $\hat{U}$ that prepares a state close to the target.

$$|\psi_{\text{out}}\rangle = \hat{U}|\psi_0\rangle, \qquad F = |\langle\psi_{\text{target}}|\hat{U}|\psi_0\rangle|^2$$

Constructing $\hat{U}$ as a sequence

$$\hat{U} = \hat{U}_{a_{m-1}} \cdots \hat{U}_{a_1} \hat{U}_{a_0}$$

Each action a_t chooses a gate from a given set, or a Hamiltonian generator $\hat{\mathcal{H}}_{a_t}$ and an allowed duration Δt_{a_t}, with

$$\hat{U}_{a_t} = e^{-i\hat{\mathcal{H}}_{a_t}\Delta t_{a_t}}, \qquad \hbar = 1.$$

We maximise the fidelity F within the available gates or evolutions and a gate or time budget. The action with index 0 acts first.

Gates Act on Quantum Amplitudes

A qubit has basis vectors $|0\rangle = (1,0)^\top$ and $|1\rangle = (0,1)^\top$. A gate is a unitary matrix acting on its amplitudes.

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}, \quad |a|^2 + |b|^2 = 1, \qquad |\psi'\rangle = \hat{U}|\psi\rangle, \quad \hat{U}^\dagger \hat{U} = \hat{I}.$$

$$\hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \hat{Y} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \hat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

- $\hat{X}$ exchanges the basis states. $\hat{Z}$ changes the relative sign of their amplitudes.
- The Pauli matrices obey $\hat{X}^2 = \hat{Y}^2 = \hat{Z}^2 = \hat{I}$, which gives simple exponential formulas for rotations.
- On a register, a gate on qubit j acts with the identity on all other qubits.

Hadamard and Phase Gates

$$\hat{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \hat{S} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad \hat{T} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

$$\hat{H}|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle, \qquad \hat{H}|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

$$\hat{T}|+\rangle = \frac{|0\rangle + e^{i\pi/4}|1\rangle}{\sqrt{2}}$$

- $\hat{H}$ creates a coherent superposition from a basis state.
- $\hat{S}$ and $\hat{T}$ change the relative phase and preserve the computational-basis probabilities. $\hat{T}^2 = \hat{S}$.

Rotations About the Pauli Axes

$$\hat{R}_\alpha(\theta) = e^{-i\theta\hat{\sigma}_\alpha/2} = c\hat{I} - is\hat{\sigma}_\alpha, \qquad c = \cos(\theta/2), \quad s = \sin(\theta/2)$$

$$\hat{R}_x(\theta) = \begin{pmatrix} c & -is \\ -is & c \end{pmatrix}, \qquad \hat{R}_y(\theta) = \begin{pmatrix} c & -s \\ s & c \end{pmatrix}$$

$$\hat{R}_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

$\hat{\sigma}_x = \hat{X}$, $\hat{\sigma}_y = \hat{Y}$, $\hat{\sigma}_z = \hat{Z}$. The factor $1/2$ makes θ the physical rotation angle.

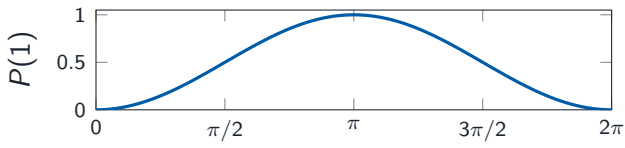
$$\hat{\mathcal{H}}_\alpha = \frac{\Omega}{2}\hat{\sigma}_\alpha, \qquad e^{-i\hat{\mathcal{H}}_\alpha \Delta t} = \hat{R}_\alpha(\Omega\Delta t)$$

Ω is the rotation rate, Δt the duration and $\hbar = 1$. In this convention $\hat{T} = e^{i\pi/8}\hat{R}_z(\pi/4)$.

The Angle Controls Amplitudes and Phase

$$\hat{R}_x(\theta)|0\rangle = c|0\rangle - is|1\rangle, \qquad \hat{R}_y(\theta)|0\rangle = c|0\rangle + s|1\rangle$$

Both give a probability $P(1) = \sin^2(\theta/2)$, while their relative phases differ.



$$\hat{R}_z(\theta)|+\rangle = \frac{e^{-i\theta/2}|0\rangle + e^{i\theta/2}|1\rangle}{\sqrt{2}}$$

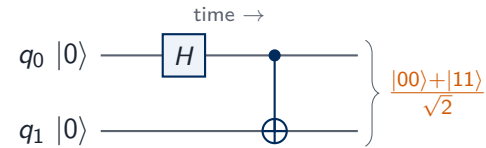
A z rotation preserves the two populations and changes the relative phase by θ. Here $c = \cos(\theta/2)$ and $s = \sin(\theta/2)$.

Controlled NOT Creates Entanglement

In the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, control first,

$$\widehat{\mathrm{CNOT}}_{0\to 1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A control in $|0\rangle$ leaves the target unchanged. A control in $|1\rangle$ applies $\hat{X}$ to the target, $|10\rangle \mapsto |11\rangle$, $|11\rangle \mapsto |10\rangle$.



The rule acts on each component of $(|00\rangle + |10\rangle)/\sqrt{2}$. The result, the Bell state, is no product of one-qubit states.

N qubits: 2^N amplitudes; single-qubit gates never create entanglement; circuits are read from left to right

Hadamard as Hamiltonian Evolution

The Hadamard gate is a Pauli operator along a tilted axis,

$$\hat{H} = \frac{\hat{X} + \hat{Z}}{\sqrt{2}}, \qquad \hat{H}^2 = \hat{I}.$$

It is a rotation through π about the axis $(\mathbf{e}_x + \mathbf{e}_z)/\sqrt{2}$, up to a global phase.

An exact exponential representation

$$\hat{H} = \exp\left[-i\frac{\pi}{2}\left(\hat{I} - \frac{\hat{X} + \hat{Z}}{\sqrt{2}}\right)\right]$$

The identity term accounts for the global phase. Choosing this generator for a specified duration implements the same gate matrix.

CNOT as Hamiltonian Evolution

The control projectors are $\hat{P}_0 = (\hat{I} + \hat{Z})/2$ and $\hat{P}_1 = (\hat{I} - \hat{Z})/2$, so

$$\widehat{\mathrm{CNOT}} = \hat{P}_0 \otimes \hat{I} + \hat{P}_1 \otimes \hat{X}.$$

$$\boxed{\widehat{\mathrm{CNOT}} = \exp\left[-i\frac{\pi}{4}(\hat{I} - \hat{Z}) \otimes (\hat{I} - \hat{X})\right]}$$

- For control 0 the generator vanishes and the target stays unchanged.
- For control 1 the target evolves under $(\pi/2)(\hat{I} - \hat{X})$, producing $\hat{X}$.

Two-Qubit Rotations

$$\hat{R}_{\alpha\alpha}(\theta) = e^{-i\theta\hat{\sigma}_\alpha\otimes\hat{\sigma}_\alpha/2} = c\hat{I}_4 - is\hat{\sigma}_\alpha\otimes\hat{\sigma}_\alpha, \quad c=\cos(\theta/2), \quad s=\sin(\theta/2).$$

$$\hat{R}_{xx}(\theta) = \begin{pmatrix} c & 0 & 0 & -is \\ 0 & c & -is & 0 \\ 0 & -is & c & 0 \\ -is & 0 & 0 & c \end{pmatrix}, \quad \hat{R}_{yy}(\theta) = \begin{pmatrix} c & 0 & 0 & is \\ 0 & c & -is & 0 \\ 0 & -is & c & 0 \\ is & 0 & 0 & c \end{pmatrix}$$

$$\hat{R}_{xx}(\theta)|00\rangle = c|00\rangle - is|11\rangle, \qquad \hat{R}_{yy}(\theta)|00\rangle = c|00\rangle + is|11\rangle.$$

Matrices use the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. At $\theta = \pi/2$ these outputs have two equal-amplitude components and are maximally entangled.

The ZZ Interaction Changes Relative Phases

$$\hat{R}_{zz}(\theta) = e^{-i\theta \hat{Z}\otimes\hat{Z}/2} = \begin{pmatrix} e^{-i\theta/2} & 0 & 0 & 0 \\ 0 & e^{i\theta/2} & 0 & 0 \\ 0 & 0 & e^{i\theta/2} & 0 \\ 0 & 0 & 0 & e^{-i\theta/2} \end{pmatrix}$$

The even-parity states $|00\rangle, |11\rangle$ and odd-parity states $|01\rangle, |10\rangle$ acquire opposite phases.

$$\hat{R}_{zz}(\theta)|++\rangle = \frac{1}{2}\left[e^{-i\theta/2}(|00\rangle + |11\rangle) + e^{i\theta/2}(|01\rangle + |10\rangle)\right]$$

At $\theta = \pi/2$ this state is entangled, while all four computational-basis probabilities remain $1/4$.

For an interaction $\hat{\mathcal{H}} = (\Omega/2)\hat{Z} \otimes \hat{Z}$, the angle is $\theta = \Omega\Delta t$. The same relation holds for the XX and YY generators.

GHZ and W States

A Greenberger–Horne–Zeilinger (GHZ) state combines two macroscopically different bit strings coherently.

$$|\mathrm{GHZ}_N\rangle = \frac{|0\ldots0\rangle + |1\ldots1\rangle}{\sqrt{2}}$$

A W state shares one excitation coherently among all positions.

$$|\mathrm{W}_N\rangle = \frac{|10\ldots0\rangle + |010\ldots0\rangle + \cdots + |0\ldots01\rangle}{\sqrt{N}}$$

- Every qubit is a two-level system with basis states $|0\rangle$ and $|1\rangle$. Here $|1\rangle$ denotes an excitation.
- These are superpositions with definite relative phases. An incoherent mixture of the displayed bit strings is a different quantum state.

Different Forms of Entanglement

GHZ

Measuring all qubits in the computational basis gives all 0s or all 1s, each with probability $1/2$.

The relative phase between the two components is a collective property. Losing one qubit removes this coherence from the remaining register, leaving a separable mixture.

W

Measuring all qubits gives one 1 at a uniformly distributed position, with probability $1/N$ per position.

Losing one qubit leaves a mixture containing the W state of the remaining qubits and the vacuum state. For $N \geq 3$, entanglement remains.

- three qubits: GHZ and W cannot be converted into each other by local operations
- W is reached exactly by $\hat{H}, \hat{S}, \hat{T}$ and CNOT only for N a power of 2

W. Dür, G. Vidal, J. I. Cirac, Phys. Rev. A 62, 062314 (2000).

Dicke States Share a Fixed Number of Excitations

A four-qubit example has two excitations in every basis component,

$$|D_4^{(2)}\rangle = \frac{1}{\sqrt{6}}(|1100\rangle + |1010\rangle + |1001\rangle + |0110\rangle + |0101\rangle + |0011\rangle).$$

Each of the six placements has the same positive amplitude. Measuring the four bits finds each placement with probability $1/6$.

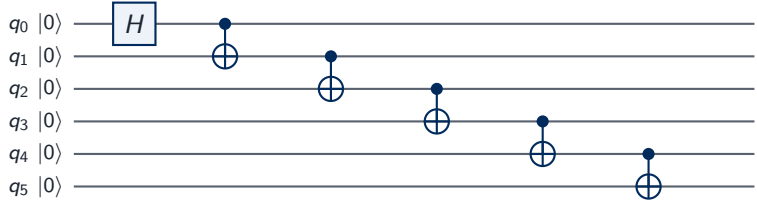
Six qubits with three excitations give $\binom{6}{3} = 20$ placements,

$$|D_6^{(3)}\rangle = \frac{|111000\rangle + |110100\rangle + \cdots + |000111\rangle}{\sqrt{20}}.$$

- The state is a coherent superposition and is unchanged by exchanging qubits.
- W shares one excitation, $|\mathrm{W}_N\rangle = |D_N^{(1)}\rangle$. Here we share three.

Preparing a GHZ State of Six Qubits

Initial state $|000000\rangle$, target $|\mathrm{GHZ}_6\rangle$, operations $\hat{H}$ and nearest-neighbour CNOT, success $F = 1$.



$\hat{H}$ creates $(|000000\rangle + |100000\rangle)/\sqrt{2}$. Each CNOT extends the correlated branch to the next qubit. Five CNOTs give $F = 1$. Resources: one $\hat{H}$ and five CNOTs, the minimum for this gate set, since every qubit must be linked by a CNOT.

The circuit acts from left to right. This known construction is a reference against which a learned circuit can be checked.

The Excitation-Transfer Gate for W Preparation

In the ordered basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$,

$$\hat{G}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c & s & 0 \\ 0 & -s & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad c = \cos\theta, \quad s = \sin\theta.$$

$$\hat{G}(\theta)|10\rangle = c|10\rangle + s|01\rangle$$

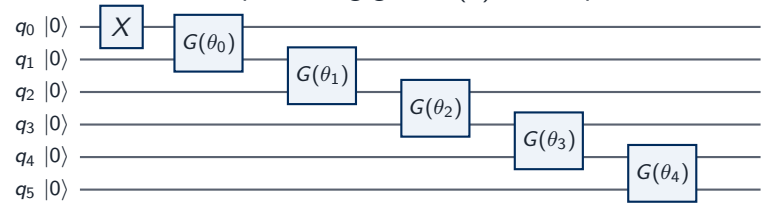
$$\hat{G}(\theta) = (\hat{I} \otimes \hat{S})\hat{R}_{xx}(\theta)\hat{R}_{yy}(\theta)(\hat{I} \otimes \hat{S}^\dagger) = \exp\Big[-\tfrac{i\theta}{2}(\hat{X} \otimes \hat{Y} - \hat{Y} \otimes \hat{X})\Big]$$

The two rotations share the excitation, and the phase gates make the transferred amplitudes real and positive in this convention.

In time order, apply $\hat{S}^\dagger$ to the second qubit, then $\hat{R}_{xx}$, $\hat{R}_{yy}$, then $\hat{S}$. The matrix leaves $|00\rangle$ and $|11\rangle$ unchanged.

Constructing the W State

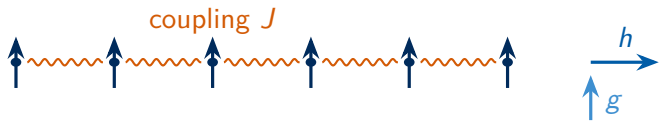
On neighbouring qubits, use the excitation-preserving gate $\hat{G}(\theta)$ of the previous frame, starting from $\hat{X}$ on q_0.



Choose $\cos\theta_j = 1/\sqrt{6-j}$, for $j = 0, \ldots, 4$. Each gate leaves amplitude $1/\sqrt{6}$ at its left qubit and passes the remaining excitation to the right.

This continuous construction reaches unit fidelity. The discrete $\hat{H}, \hat{S}, \hat{T}, \widehat{\mathrm{CNOT}}$ gate set reaches it only approximately.

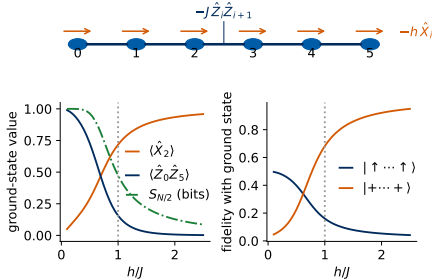
The Ising Chain



$$\hat{\mathcal{H}} = -J \sum_{i=0}^{N-2} \hat{Z}_i \hat{Z}_{i+1} - h \sum_{i=0}^{N-1} \hat{X}_i - g \sum_{i=0}^{N-1} \hat{Z}_i$$

- $\hat{X}_i$, $\hat{Z}_i$: Pauli matrices on spin i; $J > 0$: neighbours align along z, $J < 0$: they anti-align
- transverse field h turns spins towards $|+\rangle$, longitudinal field g favours one direction along z
- targets $|\psi_{\text{target}}\rangle$: ground states at $J = 1$, $h = 1$, $g = 0$, and of the longitudinal-field chain at $J = -1$, $|h| = 0.9045$, $|g| = 0.8090$, a point far from any integrable chain
- compilation uses $h, g < 0$, Hamiltonian switching $h, g > 0$

Ferromagnet and Paramagnet



six spins, $g = 0$, exact diagonalisation; left: $\langle\hat{X}_2\rangle$, $\langle\hat{Z}_0\hat{Z}_5\rangle$ and half-chain entropy; right: fidelity of two product states; dotted: $h = J$

- $h < J$: ferromagnet, finite chain close to $|\mathrm{GHZ}_N\rangle$
- $h > J$: paramagnet, close to $|+\cdots+\rangle$
- transverse-field Ising model (TFIM), $g = 0$: exactly solvable, continuous quantum phase transition at $h = J$ for $N \to \infty$

$g \neq 0$ breaks the exact solution

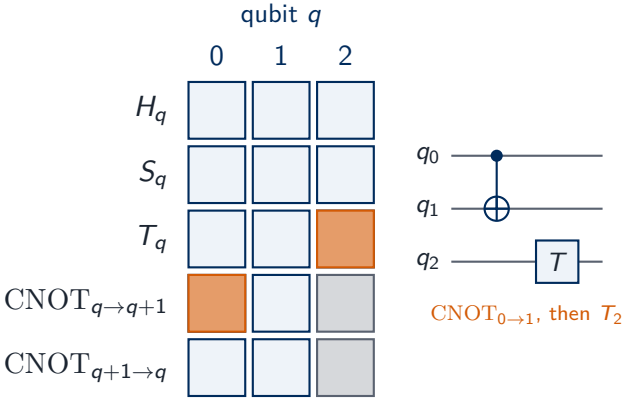
P. Pfeuty, Ann. Phys. **57**, 79 (1970); H. Kim, D. A. Huse, Phys. Rev. Lett. **111**, 127205 (2013)

State Compilation as a Decision Process

state	the current prepared state $\|\psi_t\rangle$, with the remaining budget when it affects the task
action	one allowed gate, or one allowed Hamiltonian evolution
transition	$\|\psi_{t+1}\rangle = \hat{U}_{a_t}\|\psi_t\rangle$
reward	fidelity gain minus a gate cost, $r_{t+1} = F_{t+1} - F_t - \zeta_{a_t}$
policy	a rule for choosing the next action from the current state
episode	a sequence of actions ending at success or the budget limit

We know the target. The agent must discover a sequence that prepares it using the allowed operations.

Actions Are Gates on Chosen Qubits

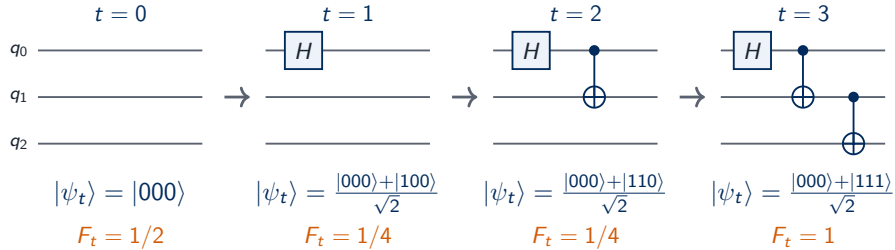


grid: the 13 actions for $N = 3$; grey: no neighbour; orange: two chosen actions

action a_t: one gate on chosen qubits

- $\hat{H}$, $\hat{S}$, $\hat{T}$ on any qubit, CNOT between neighbours in both directions
- $|\mathcal{A}| = 3N + 2(N-1) = 5N - 2$
- deterministic transition $|\psi_{t+1}\rangle = \hat{U}_{a_t}|\psi_t\rangle$

The State Is the Current Quantum State



- state $s_t = |\psi_t\rangle$, a vector of 2^N complex amplitudes
- many gate lists give one state, $\hat{H}\hat{H} = \mathbb{1}$, $\hat{S}\hat{S} = \hat{Z}$: the list is a redundant state
- network input: real and imaginary parts of the 2^N amplitudes, simulator values unavailable on a device

target $|\mathrm{GHZ}_3\rangle = (|000\rangle + |111\rangle)/\sqrt{2}$; three qubits and ten gates: 68 795 distinct states against $13^{10} \approx 1.4 \times 10^{11}$ gate lists

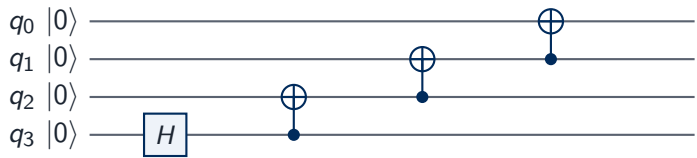
8

Learning the State-Preparation Circuit

JAGIELLONIAN UNIVERSITY
IN KRAKOW

A Learned Circuit Prepares the GHZ Target

$$|\psi_0\rangle = |0000\rangle, \qquad |\psi_{\text{target}}\rangle = \frac{|0000\rangle + |1111\rangle}{\sqrt{2}}.$$



- The Double DQN agent found this circuit in the recorded seed-0 run of Lab 052.
- Four gates, $F = 1$, cascade from the opposite end of the chain.

The Budget Changes the Continuation Value

The compiler observes the prepared state and the number h of gates still allowed.

$$
\begin{aligned}
V_{*,0}(\psi) &= 0, \\
Q_{*,h}(\psi, a) &= r(\psi, a) + \gamma V_{*,h-1}(U_a \psi), \\
V_{*,h}(\psi) &= \max_a Q_{*,h}(\psi, a).
\end{aligned}
$$

- Success and budget exhaustion have zero continuation.
- The same physical state can require different actions at different budgets.

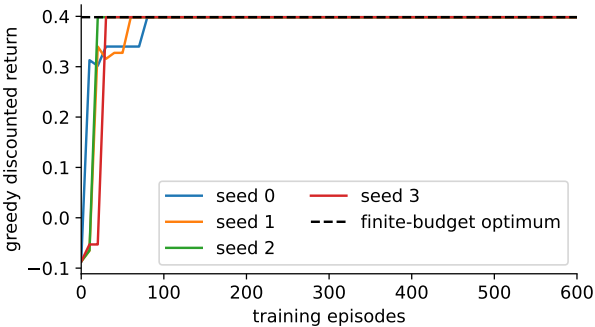
Discounted Fidelity Gains Define an Objective

With $r_{t+1} = F_{t+1} - F_t - \zeta_{a_t}$, the discounted return is

$$G_0 = -F_0 + \gamma^{n-1} F_n + \underbrace{(1-\gamma) \sum_{j=1}^{n-1} \gamma^{j-1} F_j}_{\text{intermediate fidelities}} - \underbrace{\sum_{t=0}^{n-1} \gamma^t \zeta_{a_t}}_{\text{discounted gate costs}} .$$

- At $\gamma = 1$, the fidelity terms telescope to $F_n - F_0$.
- At $\gamma < 1$, circuits with the same final fidelity can have different returns.
- Reward design specifies which circuit the learner should prefer.

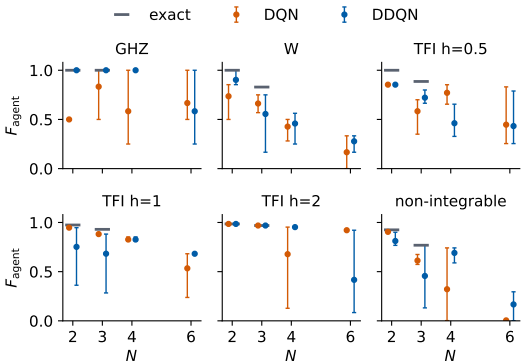
A Q-Table Learns the Restricted GHZ Circuit



Classroom study with restricted actions.

- Initial $|000\rangle$, target $|\mathrm{GHZ}_3\rangle$.
- Actions H_0, $\mathrm{CNOT}_{0\to1}$, $\mathrm{CNOT}_{1\to2}$.
- 6 gates allowed, with budget in the state.
- All 4 seeds reach return 0.39835, the backward-induction reference.

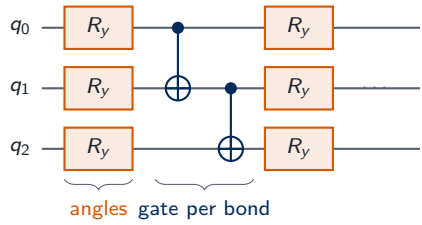
Circuit Success and Reliability Across Runs



- Curves show achieved fidelity and spread across seeds for each target family.
- A successful circuit supplies a construction. Repeated runs measure reliability.
- Six-qubit GHZ: reached by 1 of 3 DQN and 1 of 3 DDQN agents in this uniform-cost survey, by none of 5 seeds per method with the CNOT cost.

Preserved full discrete benchmark. Amplitude inputs omit remaining budget and DQN bootstraps at cutoffs.

Compilation with Continuous Angles

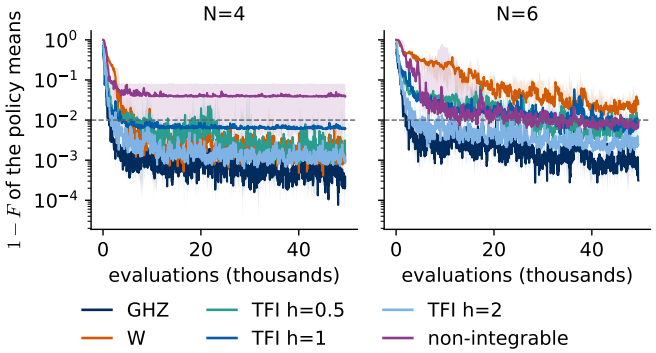


$\hat{R}_y(a) = e^{-ia\hat{\sigma}_y/2}$ with a learned angle a; one step = one layer

state	real amplitudes of $\lvert\psi_t\rangle$ and the step index t
action	N rotation angles (Gaussian) and a gate type per bond, none, CNOT or CZ (softmax)
reward	fidelity F after the last layer, 0 before
method	REINFORCE with normalised batch-centred returns

reference: gradient ascent of F through the simulator, which supplies D^*, the smallest tested depth that reaches the goal fidelity

REINFORCE Learns the Rotation Angles



infidelity $1 - F$ of the greedy circuit (mean angles, most probable gates) against the number of fidelity evaluations, median and band over three seeds; TFIM: transverse-field Ising ground state; dashed: $F = 0.99$

- interpret each target, depth and seed separately; the agent received D^* from the reference depth scan

9

Analog Control of a Spin Chain



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Actions Are Evolutions Under Hamiltonian Terms

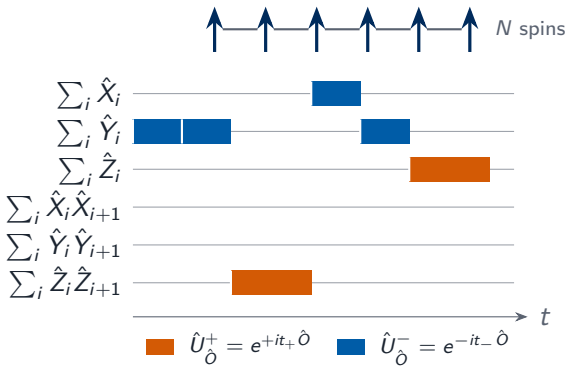


illustration of a protocol: one term $\hat{O}$ at a time, bar length = duration

- action: evolve the whole chain under one term $\hat{O}$ for a fixed time
- six terms, two signs: 12 coarse actions, $t_+ = \pi/8$, $t_- = \pi/13$
- fine versions with $t_+ = \pi/16$, $t_- = \pi/26$: 24 actions in all

the knobs of an analog simulator: uniform fields and couplings

Ground-State Preparation as a Markov Decision Process

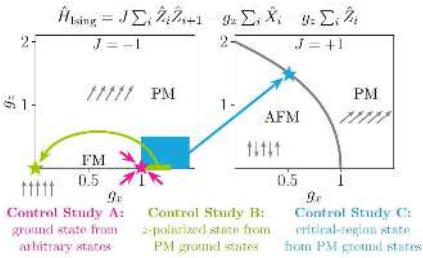


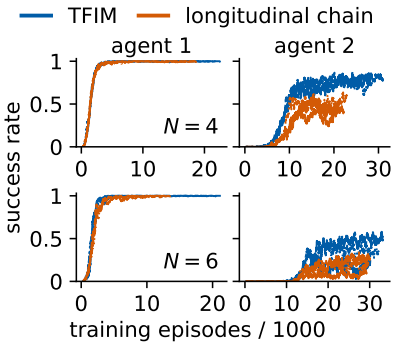
figure: Metz and Bukov, control tasks on spin chains

state	$\|\psi_t\rangle$, simulated amplitudes (2^N), $N = 4$ and 6
action	one evolution $\hat{U}_{\hat{O}}^{\pm}$
reward	$r_{t+1} = N^{-1} \ln F_{t+1} \leq 0$, zero at the target
episode	from all spins up, each tilted by a small random angle, until the threshold of the agent or 50 actions
agent	Double DQN, two hidden layers of 128 units

negative rewards favour short protocols; $\ln F/N$ stays of order one while F falls exponentially with N

F. Metz, M. Bukov, Nat. Mach. Intell. **5**, 780 (2023)

The Fine Agent Has a Harder Preparation Task



- The fine stage starts from coarse successes and must reach a higher threshold (table).

fidelity threshold	$N = 4$	$N = 6$
coarse	0.849	0.783
fine	0.968	0.953

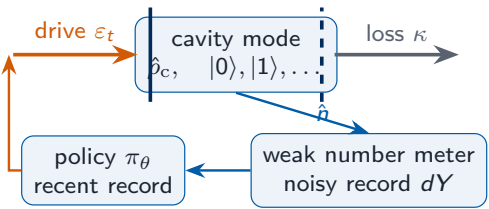
Recorded simulation; curves are individual training seeds, measured with exploration.

10

Feedback Preparation of a Photon State

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Keeping One Photon in a Leaky Cavity



- One electromagnetic mode has photon-number states $|0\rangle, |1\rangle, |2\rangle, \ldots$.
- Photons escape at a loss rate κ. A controllable classical drive supplies energy.
- The target is $\hat{\rho}_{\text{target}} = |1\rangle\langle 1|$, one photon with certainty.
- The policy chooses the drive amplitude ε_t.
- The observation is the recent noisy photon-number measurement record.

Feedback task described by Marquardt (2021) and Dawid et al., arXiv:2204.04198.

Drive, Loss and the Measurement Record

The state $\hat{\rho}_{\mathrm{c}}$ of the cavity, conditioned on the measurement record, evolves as

$$d\hat{\rho}_{\mathrm{c}} = \Big(\underbrace{-i[\hat{\mathcal{H}}_{\mathrm{d}}, \hat{\rho}_{\mathrm{c}}]}_{\text{drive}} + \underbrace{\kappa \mathcal{D}[\hat{a}]\hat{\rho}_{\mathrm{c}}}_{\text{photon loss}} + \underbrace{\Gamma_{\mathrm{m}} \mathcal{D}[\hat{n}]\hat{\rho}_{\mathrm{c}}}_{\text{measurement back-action}} \Big) dt + \underbrace{\sqrt{\Gamma_{\mathrm{m}}}\, \mathcal{M}[\hat{n}]\hat{\rho}_{\mathrm{c}}\, dW}_{\text{update by the record}},$$

$$\underbrace{dY}_{\text{measured signal}} = \underbrace{2\sqrt{\Gamma_{\mathrm{m}}}\langle \hat{n} \rangle_{\mathrm{c}}\, dt}_{\text{photon-number information}} + \underbrace{dW}_{\text{noise}}, \qquad \hat{\mathcal{H}}_{\mathrm{d}} = i\varepsilon_t(\hat{a}^\dagger - \hat{a}).$$

- $\hat{a}$ removes a photon, $\hat{n} = \hat{a}^\dagger \hat{a}$, and ε_t is the drive amplitude chosen by the controller.
- κ sets the photon-loss rate and Γ_{m} the strength of the weak photon-number measurement, which dephases without absorbing photons.
- Averaging over the record removes the last term.

$\mathcal{D}[\hat{L}]\hat{\rho} = \hat{L}\hat{\rho}\hat{L}^\dagger - \frac{1}{2}(\hat{L}^\dagger \hat{L}\hat{\rho} + \hat{\rho}\hat{L}^\dagger \hat{L})$, $\mathcal{M}[\hat{n}]\hat{\rho} = \hat{n}\hat{\rho} + \hat{\rho}\hat{n} - 2\,\mathrm{Tr}(\hat{n}\hat{\rho})\hat{\rho}$, $\langle \hat{n} \rangle_{\mathrm{c}} = \mathrm{Tr}(\hat{n}\hat{\rho}_{\mathrm{c}})$, dW a Wiener increment of variance dt.

The Cavity Reward and Learning Objective

The target population is the fidelity with the one-photon state,

$$P_1(t) = \langle 1|\hat{\rho}_c(t)|1\rangle, \qquad 0 \le P_1(t) \le 1.$$

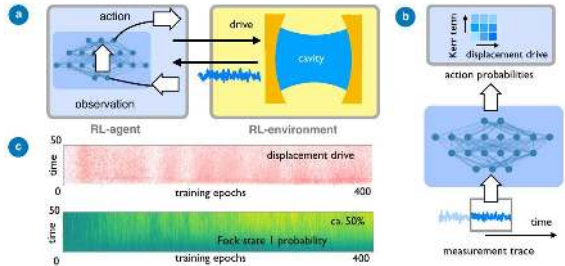
We reward the one-photon population after each control interval,

$$r_{t+1} = P_1(t+1), \qquad J(\theta) = \mathbb{E}_{\pi_\theta}\left[\frac{1}{T}\sum_{t=0}^{T-1} r_{t+1}\right].$$

We optimise the policy parameters θ, so its drive choices keep the cavity near $|1\rangle$ for as much of the episode as possible.

A mean photon number of one does not imply the state $|1\rangle$.

The Paper's Feedback-Learning Result



Panel (a) shows the drive and measurement loop. Panel (b) maps a recent measurement trace to action probabilities. Panel (c) shows the drive and one-photon probability as training progresses. The one-photon population approaches about 50% in this source example, itself a simulation; our model uses discrete Gaussian readings.

F. Marquardt, Machine learning and quantum devices, SciPost Phys. Lect. Notes 29 (2021).

11

Reading

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Reading

- R. S. Sutton and A. G. Barto, *Reinforcement Learning. An Introduction*, 2nd ed., MIT Press (2018), incompleteideas.net/book/the-book-2nd.html
- A. Dawid *et al.*, *Modern applications of machine learning in quantum sciences*, arXiv:2204.04198, chapter on reinforcement learning
- M. Bukov and F. Marquardt, *Reinforcement Learning for Quantum Technology*, arXiv:2601.18953 (2026)